DarImmo

Application mobile de services immobiliers
Recherche, consultation et publication d'annonces

Dossier de conception et de répartition du travail
Versions Flutter et React Native · Modélisation C4 · Organisation Jira

DSI2 - Développement des Systèmes Informatiques - ISET
Équipe de 3 étudiants

Version 2 (corrigée) · 4 novembre 2026
Version 1 : 15 octobre 2026

0. Historique des corrections (v1 → v2)

Section	Correction	Raison
3. Fonctionnalités	Réinitialisation du mot de passe et signalement ajoutés au MVP. Mode sombre déplacé en bonus. Filtre « nombre de pièces » intégré à la story 3.2. Back-office web déplacé en bonus.	Le dossier et le backlog ne listaient pas les mêmes fonctionnalités
4. Modèle de données	Table `profiles` liée à `auth.users`, table `device_tokens`, statuts d'annonce complets, `motif_rejet`, `bloque`, colonne PostGIS, contraintes d'unicité, dates	Champs exigés par les stories 6.5, 7.1, 7.3, 8.1, 9.2, 9.3, 3.4 absents du modèle v1
5. Choix techniques	Edge Functions pour l'envoi des push, projet Firebase obligatoire, development build Expo, choix figés (état, carte)	Supabase n'envoie pas de push FCM seul ; FCM ne marche pas dans Expo Go
6. C4	Niveau 2 : Edge Functions ajoutées, back-office web retiré du MVP. Niveau 3 : AdminModule ajouté, ApiClient redéfini	Cohérence avec le backlog (modération dans l'application mobile)
7. Planning	Planning unique, identique au backlog Jira. Flutter et React Native dans le même sprint	Le dossier (publication au Sprint 3) et le backlog (Sprint 1) se contredisaient ; une sous-tâche Jira suit toujours le sprint de sa story
9 et 11. Répartition	Nouvelles tâches 1.7 (Firebase + preuve de concept, C) et 1.8 (configuration de la carte, B). 6.5 et 7.3 avancés	Risques traités au Sprint 0, dépendance carte supprimée, charge lissée
10. Jira	Champ Assignee, import CSV, règle des sous-tâches	L'étiquette membre-X ne suffit pas pour « Assigné à moi » et les rapports
12. Charge	Totaux recalculés à partir du backlog : A 49, B 48, C 49 pts	Les totaux v1 (37 / 39 / 37) ne correspondaient pas au backlog

1. Concept et positionnement

Élément	Proposition
Nom de travail	**DarImmo** (« dar » = maison). Alternative : ImmoGo
Proposition de valeur	Trouver, publier et négocier un bien immobilier depuis un seul endroit, avec une carte interactive et une messagerie intégrée
Cibles	Chercheurs (acheteurs, locataires), annonceurs (particuliers, agences), administrateur (modération)
Différenciation	Annonces vérifiées par un modérateur, recherche autour de moi, messagerie intégrée ; en bonus : zone dessinée sur la carte, alertes personnalisées, FR / AR (RTL) / EN

2. Acteurs et rôles

Acteur	Droits
Visiteur	Parcourir et rechercher les annonces publiées, voir la carte (sans contacter)
Utilisateur inscrit	Favoris, messagerie, signalement, profil ; alertes en bonus
Annonceur	Tout ce que fait l'utilisateur, plus publier, modifier, supprimer ses annonces et suivre leur état de validation

Acteur	Droits
Administrateur	Valider ou rejeter des annonces, traiter les signalements, bloquer des comptes (écrans réservés dans l'application mobile)

> **Précision technique :** « annonceur » n'est pas un rôle stocké en base. Tout utilisateur inscrit peut publier. La colonne `profiles.role` ne contient que `utilisateur` ou `admin`.

3. Fonctionnalités par priorité

Module	MVP (obligatoire)	Version 2 (bonus)
Authentification	Inscription / connexion email, déconnexion, profil, **réinitialisation du mot de passe**	Connexion Google, vérification téléphone (OTP)
Recherche	Filtres : prix, ville, type (appartement, villa, terrain, local), vente ou location, nb de pièces	Tri (prix, date, distance), sauvegarde de recherches, recherche vocale
Détails d'une annonce	Galerie photos, description, caractéristiques, prix, contact, **signalement**	Visite virtuelle 360°, partage, calculateur de crédit
Publication	Formulaire, upload de photos, choix de la position sur carte, gestion de mes annonces, état de validation	Brouillons, boost / mise en avant, duplication, statistiques
Carte	Marqueurs des biens, « autour de moi »	Clusters, dessin de zone, itinéraire
Messagerie	Chat 1 à 1 temps réel, historique, liste des conversations avec non-lus	Envoi de photos, indicateur « lu » sous chaque message
Favoris	Ajouter / retirer, liste	Comparateur de 2 à 3 biens
Notifications	Nouveau message, changement de statut d'annonce	Alerte « nouveau bien correspondant à mes critères », baisse de prix
Administration	Modération dans l'application mobile : rôle admin, valider / rejeter, bloquer, traiter les signalements	Back-office web avec tableau de bord et statistiques
Divers	-	Mode sombre, multi-langues avec RTL, mode hors-ligne (cache)

4. Modèle de données (PostgreSQL / Supabase)

Table	Colonnes principales	Contraintes et remarques
profiles	id (uuid = auth.users.id), nom, email, telephone, photo_url, role (utilisateur | admin), bloque (bool), date_inscription	Créé par trigger à l'inscription. Le mot de passe reste géré par Supabase Auth
device_tokens NOUVEAU	id, id_user, token, plateforme (android | ios), date_maj	unique(token). Un utilisateur peut avoir plusieurs appareils
annonces	id, titre, description, type_bien (appartement | villa | terrain | local), type_offre (vente | location), prix, surface, nb_pieces, adresse, ville, latitude, longitude, **position geography(Point, 4326)**, **statut**, **motif_rejet**, id_annonceur, date_creation, **date_maj**	statut : brouillon | en_attente | publiee | rejetee | vendue | louee | masquee. Index GIST sur position, index sur (statut, ville, prix)
photos	id, id_annonce, chemin_storage, url, ordre	unique(id_annonce, ordre) ; 10 photos max ; suppression en cascade
favoris	id_user, id_annonce, date	Clé primaire (id_user, id_annonce)
conversations	id, id_annonce, id_acheteur, id_vendeur, date_creation, **date_dernier_message**	unique(id_annonce, id_acheteur) ; check(id_acheteur <> id_vendeur)
messages	id, id_conversation, id_expediteur, contenu, date_envoi, lu	Realtime activé ; trigger qui met à jour date_dernier_message
alertes	id, id_user, nom, criteres (jsonb), active	Bonus (3.6, 8.3)
signalements	id, id_annonce, id_user, motif, statut (en_attente | traite | classe), date	unique(id_user, id_annonce)

Relations : un profil publie plusieurs annonces (1-N). Une annonce a plusieurs photos (1-N). Profils et annonces sont liés par favoris et par signalements (N-N). Une conversation relie un acheteur et un vendeur autour d'une annonce et contient plusieurs messages (1-N). Un profil possède plusieurs jetons d'appareil (1-N).

Règles de sécurité (RLS) principales

Table	Lecture	Écriture
annonces	Tout le monde voit les annonces `publiee` ; l'annonceur voit toutes les siennes ; l'admin voit tout	Création / modification par l'annonceur (statut forcé à en_attente) ; changement de statut de modération par l'admin seulement
conversations, messages	Uniquement les deux participants	Uniquement les participants ; id_expediteur = utilisateur connecté
favoris, device_tokens	Le propriétaire	Le propriétaire
signalements	Auteur et admin	Création par un utilisateur connecté, traitement par l'admin
profiles	Nom et photo publics	Le propriétaire (sauf role et bloque : admin seulement)

5. Choix techniques

> **Point clé :** un seul backend Supabase partagé par les deux versions (Flutter et React Native). Le serveur n'est développé qu'une fois. Supabase ne sait pas envoyer de notification push seul : un projet Firebase et une Edge Function sont nécessaires.

Besoin	Option retenue	Alternative
Backend / BDD	**Supabase** (PostgreSQL + PostGIS, Auth, Storage, Realtime, Edge Functions)	Firebase (Auth, Firestore, Storage)
Notifications push	Firebase Cloud Messaging (API HTTP v1), appelé par une Edge Function Supabase `send-push` déclenchée par un Database Webhook	OneSignal
Carte	À figer au Sprint 0 (tâche 1.8) : Google Maps (clé API, compte de facturation requis) ou OpenStreetMap (gratuit, sans clé)	-
Géolocalisation	Flutter : geolocator / RN : expo-location	-
Stockage photos	Supabase Storage, buckets `annonces-photos` et `avatars`	Cloudinary
Configuration	Clés dans un fichier `.env` non versionné (flutter_dotenv / variables Expo)	-

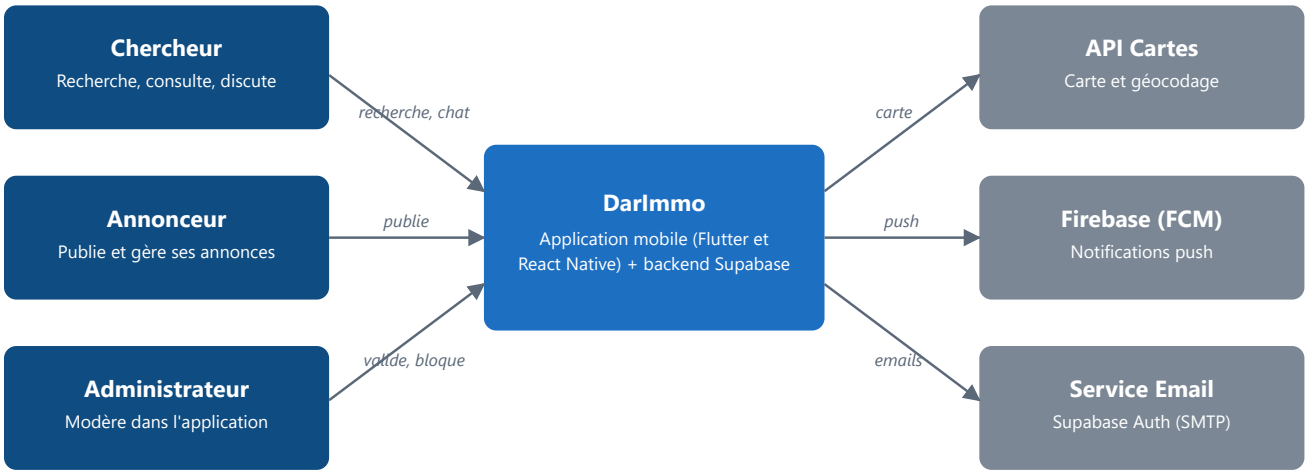
Comparatif des deux versions

Aspect	Flutter	React Native
Langage	Dart	TypeScript (Expo, development build)
Navigation	go_router	React Navigation
État	Provider (ChangeNotifier) – choix figé pour toute l'équipe	Zustand – choix figé pour toute l'équipe
Carte	google_maps_flutter ou flutter_map (selon 1.8)	react-native-maps
Backend	supabase_flutter	@supabase/supabase-js
Photos	image_picker + flutter_image_compress	expo-image-picker + expo-image-manipulator
Notifications	firebase_messaging (+ flutter_local_notifications au premier plan)	@react-native-firebase/messaging (development build obligatoire, ne marche pas dans Expo Go)

6. Modélisation C4

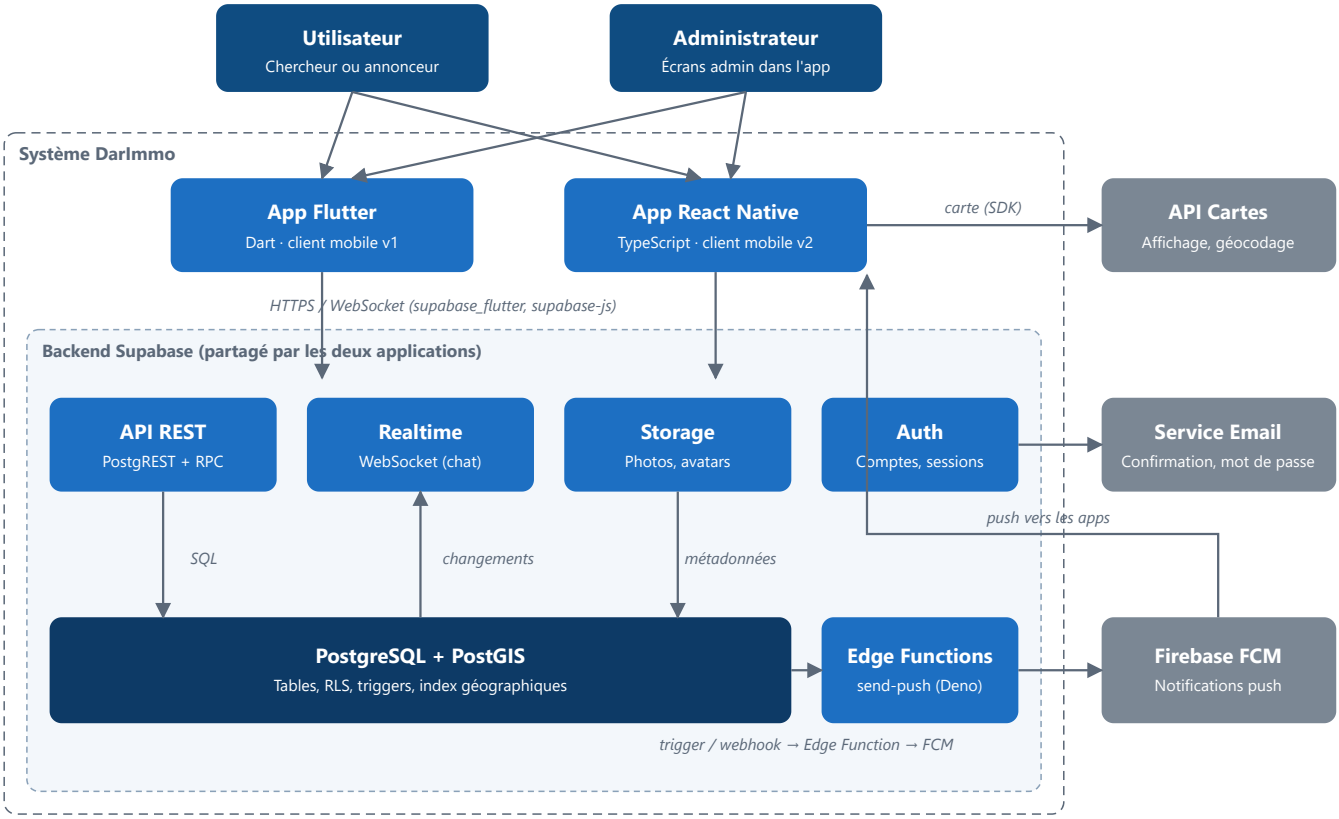
La méthode C4 (c4model.com) décrit le système en quatre niveaux de zoom : contexte, conteneurs, composants, code.

Niveau 1 : diagramme de contexte



Les trois acteurs utilisent l'application mobile. Elle s'appuie sur trois systèmes externes : cartes, notifications push (Firebase) et email (envoyé par Supabase Auth).

Niveau 2 : diagramme de conteneurs



Les deux clients mobiles partagent le même backend Supabase. L'administrateur utilise les écrans réservés de l'application mobile ; un back-office web reste un bonus. Les push passent par un trigger, une Edge Function et FCM.

Niveau 3 : composants de l'application mobile

Composant	Responsabilité	Interagit avec
AuthModule	Inscription, connexion, session, mot de passe oublié	Supabase Auth
SearchModule	Filtres, liste des résultats, pagination	SupabaseService, MapModule
ListingDetailModule	Affichage d'une annonce, galerie, signalement	SupabaseService, ChatModule, FavoritesModule
PublishModule	Formulaire, upload photos, choix de position, mes annonces, état de validation	SupabaseService, Storage, LocationService, MapModule
MapModule	Composant de carte, marqueurs, « autour de moi »	API Cartes, LocationService
ChatModule	Conversations, envoi / réception de messages	Realtime, SupabaseService
FavoritesModule	Ajout / retrait, liste des favoris	SupabaseService
ProfileModule	Profil, photo, paramètres	SupabaseService, Storage
AdminModule NOUVEAU	Annonces en attente, signalements, blocage de comptes (visible si role = admin)	SupabaseService
NotificationService	Enregistrement du jeton, réception des push, navigation vers l'écran concerné	FCM, table device_tokens
LocationService	Permissions et position GPS	Capteurs de l'appareil
SupabaseService CORRIGÉ	Point d'accès unique au client Supabase (remplace « ApiClient » : il n'y a pas d'API HTTP maison), traduction des erreurs	Supabase
LocalCache	Cache des annonces vues (bonus hors-ligne)	SQLite / Hive

Niveau 4 : code (classe clé)

Exemple pour **Annonce** : attributs de la section 4, plus `fromJson()`, `toJson()` et `copyWith()`. Elle est associée à un **AnnonceRepository** (`getAll(filters, page)`, `getById()`, `getMine()`, `create()`, `update()`, `delete()`, `uploadPhotos()`) appelé par un **AnnonceViewModel** (Provider en Flutter, store Zustand en React Native). Cette architecture (Model, Repository, ViewModel, View) s'applique aux deux versions.

```
lib/                                src/            (React Native)
  core/   (supabase_service, theme,     core/
           router, location_service)    features/
  features/                               auth/ search/ detail/ map/
    auth/ search/ detail/ map/              publish/ chat/ favorites/
    publish/ chat/ favorites/               profile/ admin/ notifications/
    profile/ admin/ notifications/       (même découpage : models,
      data/ (models, repositories)        repositories, stores, screens)
      presentation/ (viewmodels, screens, widgets)
```

7. Planning agile

Ce planning est **identique au backlog Jira v2**. Dans chaque sprint, la sous-tâche Flutter est réalisée en premier, puis la sous-tâche React Native, avant la fin du même sprint.

Sprint	Objectif	Membre A	Membre B	Membre C
0	Conception, BDD, backend, Firebase, carte, squelettes	[1.1] Diagrammes C4 niveaux 1 et 2 (3) [1.4a] Maquettes Figma : authentification, profil, administration (2) [1.5] Backend Supabase (projet, sécurité, stockage) (5) [1.6] Dépôts Git et squelettes de projet (3) **13 pts**	[1.2] Diagrammes C4 niveaux 3 et 4 (3) [1.4b] Maquettes Figma : recherche, détail, carte, favoris (2) [1.8] Choix et configuration de la carte (2) **7 pts**	[1.3] Schéma de base de données, script SQL et données de test (5) [1.4c] Maquettes Figma : publication, mes annonces, messagerie (1) [1.7] Projet Firebase et preuve de concept temps réel / push (3) **9 pts**
1	Inscription / connexion, liste, publication	[2.1] S'inscrire par email (5) [2.2] Se connecter et se déconnecter (3) [2.3] Modifier mon profil et ma photo (3) **11 pts**	[3.1] Voir la liste des annonces (5) **5 pts**	[6.1] Publier une annonce (5) **5 pts**
2	Filtres, photos, mot de passe, rôle admin	[2.4] Réinitialiser mon mot de passe (3) [9.1] Rôle administrateur (2) **5 pts**	[3.2] Filtrer les annonces (8) **8 pts**	[6.2] Ajouter des photos (5) **5 pts**
3	Détail, mes annonces, état de validation, tests auth/recherche	[10.1] Tests unitaires et d'intégration : authentification et recherche (3) **3 pts**	[3.3] Consulter le détail d'une annonce (5) **5 pts**	[6.3] Gérer mes annonces (3) [6.5] Voir l'état de validation de mes annonces (2) **5 pts**
4	Carte, autour de moi, position du bien, contact, conversations, validation admin	[9.2] Valider ou rejeter une annonce (5) **5 pts**	[4.1] Voir les annonces sur une carte (5) [4.2] Voir les biens autour de moi (5) **10 pts**	[6.4] Placer mon bien sur la carte (3) [7.1] Contacter l'annonceur depuis une annonce (3) [7.3] Voir mes conversations (3) **9 pts**
5	Chat temps réel, favoris, signalement, blocage	[9.3] Bloquer ou débloquer un compte (3) **3 pts**	[3.4] Signaler une annonce (3) [5.1] Ajouter ou retirer une annonce des favoris (3) [5.2] Consulter mes favoris (2) **8 pts**	[7.2] Échanger en temps réel (8) **8 pts**
6	Notifications, signalements, tests, finitions	[9.4] Traiter les signalements (3) [10.2] Tests unitaires et d'intégration : publication, messagerie, notifications (3) **6 pts**	*appui, bonus, corrections* **0 pts**	[8.1] Être notifié d'un nouveau message (5) [8.2] Être notifié du changement de statut de mon annonce (3) **8 pts**

Sprint	Objectif	Membre A	Membre B	Membre C
7	Rapport comparatif, soutenance	[10.4] Préparation de la soutenance et de la démo (3) **3 pts**	[10.3] Rapport comparatif Flutter vs React Native (5) **5 pts**	*appui, bonus, corrections* **0 pts**

> **Charge réelle :** chaque story est codée deux fois. Si les sprints d'une semaine sont trop courts, passer à des sprints de deux semaines plutôt que de séparer Flutter et React Native : en Jira, une sous-tâche suit toujours le sprint de sa story parente.

8. Stratégie de tests

Type	Outils et contenu
Unitaires	flutter_test / Jest : modèles (fromJson / toJson), validations de formulaire, ViewModels
Widgets / composants	flutter_test / React Native Testing Library
Intégration / E2E	integration_test / Detox (ou Maestro)
Sécurité de la base	Scénarios SQL : un utilisateur ne lit pas les messages d'une autre conversation, ne modifie pas l'annonce d'un autre, un visiteur ne voit pas les annonces en attente
Manuels	Scénarios : créer un compte, publier, faire valider par l'admin, chercher, discuter, recevoir une notification. Données de test fournies par le script seed (1.3)

9. Répartition à 3 membres

Chaque version (Flutter et React Native) doit être maîtrisée par tous pour la soutenance. La répartition se fait donc **par modules fonctionnels** : chacun développe ses modules dans les deux frameworks.

Membre	Modules	Responsabilités transverses
A	Authentification, profil, rôles, modération dans l'app, mise en place de Supabase	Scrum Master, tests, dépôts Git
B	Recherche, filtres, détails, signalement, favoris, carte, géolocalisation, configuration de la carte	Product Owner, cohérence des maquettes, rapport comparatif Flutter vs RN
C	Publication, upload photos, mes annonces, messagerie, notifications, Firebase	Tech Lead BDD (schéma, script SQL, RLS de ses tables, données de test)

Répartition de la conception

Livrable	Responsable	Relecture
C4 niveaux 1 et 2	A	B
C4 niveaux 3 et 4	B	C
Schéma de BDD, script SQL, RLS, données de test	C	A
Projet Firebase et preuve de concept temps réel / push	C	A
Choix et configuration de la carte	B	C
Maquettes : chacun dessine les écrans de ses modules	A, B, C	B (cohérence)

10. Configuration de Jira

Étape	Action
1	Créer un compte sur atlassian.com, puis un site Jira (gratuit jusqu'à 10 utilisateurs)
2	**Créer un projet** : modèle **Scrum**, nom *DarImmo*, clé **IMM**
3	**Inviter** les 2 autres membres
4	Colonnes du tableau : À faire, En cours, En revue, Terminé
5	Composants : Flutter, React Native, Backend, Design, Documentation
6	Étiquettes : mvp, bonus, bug, urgent, bloqué, membre-A / B / C
7	**Importer** le fichier `DarImmo_backlog_jira_v2.csv` (Paramètres Jira, Système, Importation externe, CSV) ou saisir les tickets à la main
8	Renseigner le champ **Assignee** de chaque ticket NOUVEAU : l'étiquette membre-X ne suffit pas pour le filtre « Assigné à moi » et les rapports
9	Créer Sprint 0 à Sprint 7 et y glisser les stories. Les sous-tâches suivent automatiquement le sprint de leur story
10	Activer l'**intégration GitHub** pour lier commits et tickets

Conventions à adopter

Sujet	Règle
Format d'une story	« En tant que [rôle], je veux [action] afin de [bénéfice] »
Estimation	Story points (1, 2, 3, 5, 8), estimés ensemble (planning poker)
Branches Git	feature/IMM-12-login-flutter, feature/IMM-12-login-rn (IMM-12 = clé Jira réelle après import)

Sujet	Règle
Messages de commit	IMM-12: ajout écran de connexion
Revue de code	A relit B, B relit C, C relit A (une Pull Request par sous-tâche)
Definition of Done	Fonctionne sur émulateur, critères d'acceptation vérifiés, testé, relu, mergé dans develop, ticket en « Terminé »
Durée de sprint	1 semaine (2 si les cours sont chargés), identique pour toute l'équipe

Rituels

Rituel	Fréquence	Durée	Contenu
Sprint Planning	Début de sprint	30 min	Choisir les stories, estimer
Daily (asynchrone WhatsApp / Discord)	Chaque jour	5 min	Fait hier / prévu aujourd'hui / blocages
Sprint Review	Fin de sprint	20 min	Démo de ce qui fonctionne
Rétrospective	Fin de sprint	15 min	Ce qui va, ce qui ne va pas, actions

11. Backlog MVP (résumé du backlog Jira v2)

Chaque story a deux sous-tâches (Flutter et React Native). Les points correspondent à la story entière. Le détail des critères d'acceptation se trouve dans le backlog Jira v2.

Epic 1 : Conception et mise en place

Story / tâche	Resp.	Pts	Sprint
[1.1] Diagrammes C4 niveaux 1 et 2 CORRIGÉ	A	3	0
[1.2] Diagrammes C4 niveaux 3 et 4	B	3	0
[1.3] Schéma de base de données, script SQL et données de test CORRIGÉ	C	5	0
[1.4a] Maquettes Figma : authentification, profil, administration	A	2	0
[1.4b] Maquettes Figma : recherche, détail, carte, favoris	B	2	0
[1.4c] Maquettes Figma : publication, mes annonces, messagerie	C	1	0
[1.5] Backend Supabase (projet, sécurité, stockage) CORRIGÉ	A	5	0
[1.6] Dépôts Git et squelettes de projet	A	3	0
[1.7] Projet Firebase et preuve de concept temps réel / push NOUVEAU	C	3	0
[1.8] Choix et configuration de la carte NOUVEAU	B	2	0

Epic 2 : Authentification et profil

Story / tâche	Resp.	Pts	Sprint
[2.1] S'inscrire par email	A	5	1
[2.2] Se connecter et se déconnecter	A	3	1
[2.3] Modifier mon profil et ma photo	A	3	1
[2.4] Réinitialiser mon mot de passe	A	3	2

Epic 3 : Recherche et consultation

Story / tâche	Resp.	Pts	Sprint
[3.1] Voir la liste des annonces CORRIGÉ	B	5	1
[3.2] Filtrer les annonces CORRIGÉ	B	8	2
[3.3] Consulter le détail d'une annonce	B	5	3
[3.4] Signaler une annonce	B	3	5

Epic 4 : Carte et géolocalisation

Story / tâche	Resp.	Pts	Sprint
[4.1] Voir les annonces sur une carte	B	5	4
[4.2] Voir les biens autour de moi	B	5	4

Epic 5 : Favoris

Story / tâche	Resp.	Pts	Sprint
[5.1] Ajouter ou retirer une annonce des favoris	B	3	5
[5.2] Consulter mes favoris	B	2	5

Epic 6 : Publication d'annonces

Story / tâche	Resp.	Pts	Sprint
[6.1] Publier une annonce CORRIGÉ	C	5	1
[6.2] Ajouter des photos CORRIGÉ	C	5	2
[6.3] Gérer mes annonces CORRIGÉ	C	3	3
[6.4] Placer mon bien sur la carte CORRIGÉ	C	3	4
[6.5] Voir l'état de validation de mes annonces CORRIGÉ	C	2	3

Epic 7 : Messagerie

Story / tâche	Resp.	Pts	Sprint
[7.1] Contacter l'annonceur depuis une annonce	C	3	4
[7.2] Échanger en temps réel	C	8	5
[7.3] Voir mes conversations CORRIGÉ	C	3	4

Epic 8 : Notifications

Story / tâche	Resp.	Pts	Sprint
[8.1] Être notifié d'un nouveau message CORRIGÉ	C	5	6
[8.2] Être notifié du changement de statut de mon annonce	C	3	6

Epic 9 : Administration et modération

Story / tâche	Resp.	Pts	Sprint
[9.1] Rôle administrateur	A	2	2
[9.2] Valider ou rejeter une annonce	A	5	4
[9.3] Bloquer ou débloquer un compte	A	3	5
[9.4] Traiter les signalements	A	3	6

Epic 10 : Qualité et livrables

Story / tâche	Resp.	Pts	Sprint
[10.1] Tests unitaires et d'intégration : authentification et recherche	A	3	3
[10.2] Tests unitaires et d'intégration : publication, messagerie, notifications	A	3	6
[10.3] Rapport comparatif Flutter vs React Native	B	5	7
[10.4] Préparation de la soutenance et de la démo	A	3	7

Stories bonus (sans sprint, à planifier si le MVP est terminé)

Story	Resp.	Pts
[2.5] Connexion Google et vérification du téléphone (OTP)	A	5
[2.6] Mode sombre NOUVEAU	A	2
[3.5] Trier les résultats	B	3
[3.6] Sauvegarder une recherche	B	5
[3.7] Recherche vocale	B	3
[3.8] Partager une annonce	B	2

Story	Resp.	Pts
[3.9] Calculateur de crédit	B	3
[3.10] Visite virtuelle 360°	B	5
[4.3] Regrouper les marqueurs (clusters)	B	3
[4.4] Dessiner une zone de recherche	B	8
[4.5] Itinéraire vers un bien	B	3
[5.3] Comparer plusieurs biens	B	5
[6.6] Brouillons d'annonces	C	3
[6.7] Dupliquer une annonce	C	2
[6.8] Mettre une annonce en avant (boost)	C	5
[6.9] Statistiques de mes annonces	C	5
[7.4] Envoyer des photos dans le chat	C	5
[7.5] Indicateur « lu »	C	3
[8.3] Alerte nouveau bien selon mes critères	C	8
[8.4] Alerte de baisse de prix	C	3
[9.5] Tableau de bord de statistiques (back-office web)	A	5

12. Dépendances et charge par membre

Dépendances à surveiller

Dépendance	Solution
Tous dépendent du schéma de BDD et du backend	Script SQL de C (1.3) puis configuration de A (1.5) au **Sprint 0**
B (3.1) et C (6.1) ont besoin de l'authentification de A, réalisée au même Sprint 1	Comptes de test créés à la main dans Supabase et données de test (seed) en attendant
« Placer mon bien sur la carte » (C, 6.4) et la carte (B, 4.1) sont au Sprint 4	Le composant de carte commun est livré au Sprint 0 (1.8, B) : plus de dépendance dans le sprint
« Contacter l'annonceur » (C, 7.1) a besoin de l'écran de détail (B, 3.3)	3.3 au Sprint 3, 7.1 au Sprint 4
Validation par l'admin (A, 9.2) et notification de statut (C, 8.2)	9.2 au Sprint 4, 8.2 au Sprint 6 ; 6.5 se teste avant en modifiant le statut à la main
Traitement des signalements (A, 9.4) a besoin du signalement (B, 3.4)	3.4 au Sprint 5, 9.4 au Sprint 6
Notifications push (C, 8.1 et 8.2)	Risque levé dès le Sprint 0 par la preuve de concept 1.7
Conflits Git sur les squelettes	Un dossier par module (features/auth, features/publish...), fichiers partagés (router, thème) modifiés par Pull Request

Charge par membre (story points MVP)

Membre	Détail	Total
A	Epic 1 : 13 pts · Epic 2 : 14 pts · Epic 9 : 13 pts · Epic 10 : 9 pts	**49**
B	Epic 1 : 7 pts · Epic 3 : 21 pts · Epic 4 : 10 pts · Epic 5 : 5 pts · Epic 10 : 5 pts	**48**
C	Epic 1 : 9 pts · Epic 6 : 18 pts · Epic 7 : 14 pts · Epic 8 : 8 pts	**49**

Les totaux sont équilibrés. À ajuster en planning poker, en déplaçant une story d'un membre à l'autre si besoin. Les stories bonus ne sont pas comptées.

Vues Jira utiles

Vue	Usage
Tableau (Board)	Suivi quotidien
Backlog	Planification des sprints
Chronologie (Timeline)	Vue d'ensemble des epics, utile pour la soutenance
Rapports, Burndown	Preuve d'avancement agile
Filtre « Assigné à moi »	Chacun voit ses tâches (nécessite le champ Assignee)